Programación Concurrente y Paralela

Departamento de Computación – FCEyN – UBA

Segundo cuatrimestre de 2026

Simulacro 2do. parcial – 2do. cuat. de 2026

Nombre y apellido: _____

Nº orden: _____ L.U.: _____ Cant. hojas: ____

1	2	3	4	Nota

- Cada ejercicio debe realizarse en hojas separadas y numeradas. Debe identificarse cada hoja con nombre, apellido, LU y número de orden. Hojas sin identificar no serán corregidas.
- Entregue esta hoja junto al examen, la misma **no** se incluye en la cantidad total de hojas entregadas.
- **Cada código o pseudocódigo debe estar bien explicado y justificado. ¡Obligatorio!**
- Toda suposición o decisión que tome deberá justificarla adecuadamente. Si la misma no es correcta o no se condice con el enunciado no será tomada como válida y será corregida acorde.
- La devolución de los exámenes corregidos es personal. Los pedidos de revisión se realizarán por escrito, antes de retirar el examen corregido del aula.
- Los parciales tienen dos notas: I (Insuficiente) y A (Aprobado).
- Para tener la nota aprobada es necesario tener al menos dos ejercicios calificados como bien y el restante al menos regular. Además, alguno de los ejercicios calificados como bien debe ser el de STM o el de mensajes.

Ejercicio 1: Centro de atención. Una empresa de internet tiene una línea de atención al cliente que colapsa en cada corte masivo, cuando a las consultas de siempre se suman todos los que se quedaron sin servicio. Para el próximo corte, decidió atender con agentes de IA, y antes de ponerlos a funcionar quiere simular la línea con STM.

En la simulación, las llamadas son de dos tipos, urgentes (clientes sin servicio) y comunes (consultas de facturación). Cada tipo tiene su propia cola de espera, con lugar para K llamadas. Cada cliente que llama es un thread que ejecuta `llamar` y deja su llamada en la cola del tipo que corresponde. Si la cola está llena, el cliente espera a que se libere un lugar. Una vez en la cola, espera en la línea hasta que un agente toma su llamada, y recién ahí `llamar` retorna.

Cada agente es un thread que ejecuta `agente`. Atiende de a una llamada por vez y siempre prefiere una urgente, si hay. Para que las comunes no queden relegadas, un agente que atendió 3 urgentes seguidas tiene que atender una común antes de volver a una urgente, si hay alguna común esperando.

Se cuenta con lo siguiente:

```
1  data Tipo = Urgente | Comun
2  type Llamada = Int
3  atender :: Llamada -> IO ()   -- El agente atiende la llamada
```

Se cuenta además con el siguiente esqueleto:

```
1  data Centro = ...
2
3  nuevoCentro :: Int -> IO Centro
4  nuevoCentro k = ...
5
6  llamar :: Centro -> Tipo -> Llamada -> IO ()
7  llamar centro tipo llamada = ...
8
9  agente :: Centro -> IO ()
10 agente centro = ...
```

a) Complete el esqueleto usando STM, incluida la representación del centro. La solución no debe tener deadlocks ni race conditions. Justifique todas las decisiones tomadas.

b) Indique si su solución puede sufrir starvation, justificando a partir de cómo implementa Haskell las transacciones. Si puede, describa una traza que lleve a esa situación.

c) La empresa contrata un centro tercerizado como respaldo, con sus propias colas y agentes. Todos los clientes aceptan que los atienda el primero de los dos centros que tome su llamada. Explique, sin escribir código, cómo adaptaría su solución para estos clientes. Justifique.

Ejercicio 2: Trapitos. Si uno va a un partido de fútbol en auto, no puede escaparle a una constante en la vida del hincha: los trapitos. Pero estamos en el siglo XXI, la barra se modernizó y quiere un simulador para probar qué tan efectivos son distintos modelos de organización. A cambio de un palco vitalicio, la barra nos encargó que simulemos el siguiente sistema de trapitos y lugares.

Los lugares son finitos y están contados. La barra controla N lugares de estacionamiento que se alquilan a los autos que van llegando. ¿Cómo se hace eso?

Cada hincha motorizado que llega pide un lugar y espera. Un trapito al azar (hay una cantidad arbitraria) toma el pedido y busca un lugar libre donde estacionar. Si no hay, espera; si hay, reserva el lugar, le avisa a la barra que ese lugar fue reservado y le avisa al hincha qué lugar le fue asignado. La barra, que es única, registra la reserva y cada 5 minutos suma 1 al costo del alquiler del lugar. Arranca en 1.

Al terminar el partido (o al querer irse, si se va antes de que termine), el hincha le pregunta a la barra cuánto tiene que pagar e intenta pagar ese monto. Si en el medio pasaron otros 5 minutos y el precio aumentó, tiene que volver a consultar y reintentar, porque no puede retirar el auto mientras pague menos de lo debido. Cuando el pago alcanza, la barra lo deja retirarse y libera el lugar para otro hincha.

Implementar utilizando comunicación por medio de mensajes, con las reglas vistas en clase[1]:

a) Los threads `trapito`, `hincha` y `barra`. Habrá T trapitos, H hinchas y 1 barra. Pueden usar funciones y threads auxiliares. Para simular al hincha viendo el partido, llamen a una función imaginaria `verPartido()` después de que el hincha haya estacionado y antes de retirar el auto. La solución debe ser libre de deadlocks y se tiene que justificar. Se valora el uso de diagramas para dicha explicación.

b) *(Esta parte NO entra en el parcial, pero la pusimos porque nos gusta la discusión.)* ¿Qué pasaría si, en vez de una barra, hubiera muchas? ¿Qué cambios harían?

Ejercicio 3. Un servidor atiende a dos clientes. El cliente i manda sus pedidos por el canal $\texttt{pedido}_i$ y recibe las respuestas por $\texttt{resp}_i$, todos de tipo `chan bool` y sin buffer. Se consideran dos versiones del servidor, A y B:

```
for {  // cliente 1
  pedido1 <- true
  <-resp1
  usar()
}
```

```
for {  // cliente 2
  pedido2 <- true
  <-resp2
  usar()
}
```

```
// servidor, versión A
for {
  select {
  case <-pedido1: resp1 <- true
  case <-pedido2: resp2 <- true
  }
}
```

```
// servidor, versión B
for {
  <-pedido1; resp1 <- true
  <-pedido2; resp2 <- true
}
```

a) Modele las dos versiones del sistema como composición de LTS. Indique en qué estados valen las proposiciones $pide_i$, $espera_i$, usa_i y $atiende_i$, que se usan en los ítems siguientes.

b) Escriba en LTL las siguientes propiedades:

1. Todo cliente que pide termina usando una respuesta.
2. Un cliente que está esperando su respuesta no vuelve a pedir sin haberla usado antes.
3. Entre dos atenciones al cliente 1, el servidor atiende al cliente 2.

c) Determine si cada versión satisface cada una de las siguientes fórmulas. Para las que no, dé una traza que la viole y decida si pasa a valer bajo fair choice.
1. $\square\lozenge usa_2$ 2. $\square(pide_1 \to \lozenge espera_1)$ 3. $\square\neg(usa_1 \wedge usa_2)$ 4. $\square(espera_1 \to \bigcirc(espera_1 \vee usa_1))$

Ejercicio 4.

Consideramos dos algoritmos *divide and conquer* secuenciales para calcular la función `maxSubArray` que, dado un arreglo de enteros `A` de longitud n (puede asumir n potencia de 2), determina el valor de la máxima suma de un subarreglo contíguo en A. Es decir, si por ejemplo tenemos el arreglo

$$[-2, 1, -3, 4, -1, 2, 1, -5, 4]$$

la suma máxima es 6, dada por el subarreglo $[4, -1, 2, 1]$. Se pide:

[1] Sin memoria compartida y con el `nil` propio de cada tipo. Los canales se comportan igual que en Go.

a) Adapte el pseudocódigo de estos algoritmos para que trabajen bajo un esquema *fork-join*.

b) Para ambos, calcule el *work*, el *span*, y el paralelismo.

c) En función de los resultados obtenidos, concluya cuál de las dos implementaciones es más conveniente (desde el punto de vista teórico) para su uso en una computadora de alta capacidad de paralelismo (explique el *speedup* alcanzable, por ejemplo, para una entrada de tamaño 2^{10} en una computadora con 64 procesadores, basándose en la aproximación $T_P \approx T_1/P + T_\infty$).

Resolvemos este problema de las siguientes maneras:

a) Variante (I):

```
int maxSubArray(int[] A, int lo, int hi) {
    if (hi - lo == 1) {
        return A[lo];
    }
    int mid = (lo + hi) / 2;
    int low = maxSubArray(A, lo, mid);
    int high = maxSubArray(A, mid, hi);
    int cruza = mejorSumaLineal(A, mid-1, lo-1)  // mejor sufijo de la parte baja
              + mejorSumaLineal(A, mid, hi-1);   // mejor prefijo de la parte alta
    return max(low, high, cruza);
}
```

Donde la respuesta es `maxSubArray(A, 0, len(A)-1)`. La función `mejorSumaLineal(A, desde, hasta)` recorre *secuencialmente*, en $\Theta(|desde - hasta|)$ pasos, las posiciones desde `desde` hacia `hasta`, y devuelve la máxima suma de un segmento que arranca en la posición `desde` y avanza hacia `hasta` (por lo que hacia la derecha da el mejor prefijo y hacia la izquierda da el mejor sufijo del subarreglo).

b) Variante (II):

```
Tupla mejoresCasos(int[] A, int lo, int hi) {
    // Tupla(total, optimo, prefijo, sufijo)
    if (hi - lo == 1) {
        return new Tupla(A[lo], A[lo], A[lo], A[lo]);
    }
    int mid = (lo + hi) / 2;
    Tupla b = mejoresCasos(A, lo, mid);
    Tupla d = mejoresCasos(A, mid, hi);
    int total   = b.total + d.total;
    int optimo  = max(b.optimo, d.optimo, b.sufijo + d.prefijo);
    int prefijo = max(b.prefijo, b.total + d.prefijo);
    int sufijo  = max(d.sufijo, d.total + b.sufijo);
    return new Tupla(total, optimo, prefijo, sufijo);
}
```

Donde la respuesta es `mejoresCasos(A, 0, len(A)-1).optimo`.